

KV Cache & LLM Inference

From First Principles to Production Sizing

You already understand model-size math, quantization, parallelism, batching, and rough GPU-fleet estimation. This document goes after the one thing your earlier walkthrough hand-waved as “~1 GB per user”: **the KV cache itself** — derived term by term, every type explained with the arithmetic, so you can re-derive it on a whiteboard and teach it to anyone.

[Why KV cache exists](#)[The memory formula, derived](#)[MHA → MQA → GQA](#)[KV quantization](#)[PagedAttention](#)[Full GPU budget](#)[Prefill vs decode](#)[Production architecture](#)[Self-test questions](#)

The one sentence to remember. Model weights are a *fixed* cost shared by everyone; the KV cache is a *per-user, per-token* cost that nobody shares — and in production it is almost always the KV cache, not the weights, that decides how many users a GPU can serve.

Correction carried through this doc. Your earlier notes labeled “Layer 1–40 on GPU1, Layer 41–80 on GPU2” as **Tensor Parallelism**. That layout is actually **Pipeline Parallelism**. Tensor parallelism splits the math *inside* each layer. The distinction changes how KV cache is divided across GPUs, so we fix it properly in §7.

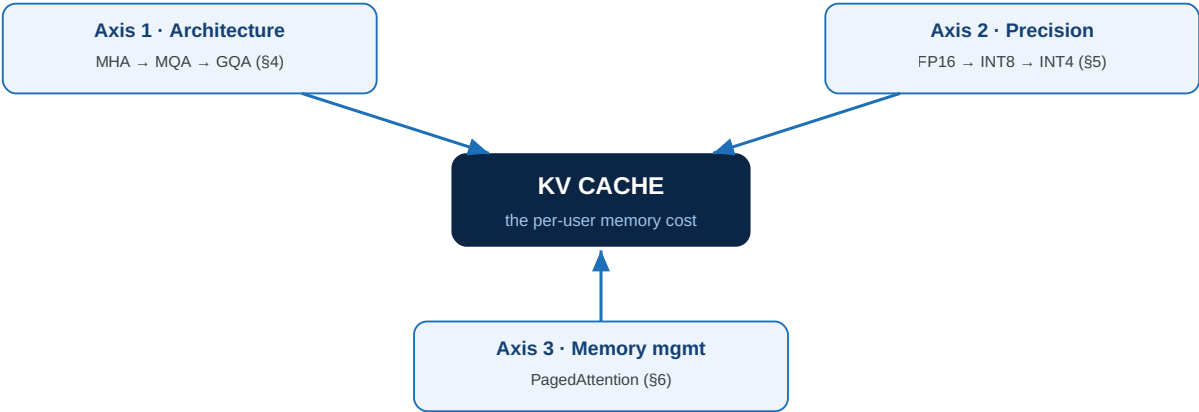
Worked examples anchored to **NVIDIA H100 (80 GB)** and **Llama-3-70B** · All numbers derived, not asserted · Estimates for provider GPU fleets are illustrative, not official.

0 · The map: four memory costs, three KV axes

Every "will this run?" question reduces to one inequality:

$$\text{GPU memory needed} = \text{Weights} + \text{KV Cache} + \text{Activations} + \text{Overhead} \leq \text{Total VRAM}$$

Your earlier walkthrough nailed the **Weights** term. This document fills in the **KV Cache** term completely, then reassembles the whole inequality in §7. The KV cache can be attacked along exactly three independent axes — keep these straight and everything else falls into place:



The three independent levers on KV-cache cost. They compose: $GQA \times INT8 \times \text{paging}$ multiply together.

§	Section	What you walk away able to do
1	Foundations recap (fast)	State the full inference-memory inequality, not just weights
2	Why KV cache exists	Explain the $O(n^2) \rightarrow O(n)$ trick from the attention mechanism
3	The KV cache formula	Derive bytes-per-token and bytes-per-sequence for any model
4	MHA / MQA / GQA	Show why GQA cuts cache 8× with one number change
5	KV quantization	Stack precision savings on top of architecture savings
6	PagedAttention	Explain why naïve allocation wastes 60–80% of cache memory
7	Full GPU budget	Compute exactly how many users a 2×/4×H100 box holds
8	Prefill vs decode	Explain why decode is memory-bound and KV-cache-limited
9	Production architecture	Draw the end-to-end serving pipeline from memory

1 · Foundations recap — and the gap to close

You have this part. We move fast and then expose the gap. A **parameter** is one learned number; storing it costs a fixed number of bytes set by its **datatype**:

Datatype	Bits	Bytes/param	Typical use
FP32	32	4	Reference / some training
FP16 / BF16	16	2	Standard inference & training
INT8 / FP8	8	1	Quantized serving
INT4	4	0.5	Aggressive quantized serving

$$\text{Weight memory} = N_{\text{params}} \times \text{bytes_per_param}$$

Llama-3-70B in FP16: $70 \times 10^9 \times 2 \text{ bytes} = \mathbf{140 \text{ GB}}$. That is the number your earlier walkthrough used. But here is the gap it never closed:

The 140 GB is weights only. It tells you nothing about how many *users* you can serve. Two boxes can both hold the same 140 GB of weights yet one serves 4 users and the other serves 70 — the difference is entirely in the KV-cache term, which the weight formula is blind to. Everything from §2 onward is about quantifying that blind spot.

Mental model. Weights = the *restaurant kitchen* (built once, fixed size, serves everyone). KV cache = the *tables* (each diner occupies one for the length of their meal). You can have a huge kitchen and still turn people away because every table is full. Production capacity is a table-counting problem.

2 · Why the KV cache exists at all

To size the cache you must know *what* is being cached and *why*. That comes straight from the attention mechanism.

2.1 Q, K, V in one paragraph

Inside every attention layer, each token is turned into three vectors by three learned weight matrices:

- **Query (Q)** — "what am I looking for?" (the current token asking).
- **Key (K)** — "what do I offer as a match target?" (every token advertises one).
- **Value (V)** — "what content do I hand over if matched?" (every token carries one).

A token attends by comparing its **Q** against the **K** of all earlier tokens, turning those scores into weights, and taking a weighted sum of their **V**. The output for position i depends on the K and V of every position $\leq i$.

2.2 The wasteful way (no cache): $O(n^2)$

Generation is autoregressive — one token at a time, each appended and fed back in. Naïvely, to produce token 1000 you re-run the whole prompt of 999 tokens through every layer, recomputing all their K and V. To produce token 1001 you redo all 1000. The K/V for token 5 gets recomputed roughly 995 times.

WITHOUT cache — recompute everything each step ($O(n^2)$ total)

step 3:  recompute K,V for t1,t2,t3

step 4:  recompute t1,t2,t3,t4 AGAIN

step 5:  ... and again

WITH cache — compute each token's K,V exactly once ($O(n)$ total)

step 5:  reuse cached t1-t4 (green), compute only t5 (solid)

The cache converts quadratic recomputation into linear work: store each token's K and V once, reuse forever.

2.3 The fix: cache K and V (not Q)

Since a past token's K and V never change once computed, we store them. Each new step computes K,V *only* for the one new token, appends them, and attends over the whole stored set. **Q is not cached** — each step only needs the current token's query; past queries are never reused.

This is the entire reason the KV cache exists: trade memory for compute. We spend VRAM holding past K and V so we never recompute them. The cost of that trade — in bytes — is the whole subject of §3.

Check yourself (answer in §11): Why don't we cache Q as well? What property of K and V makes them cacheable that Q lacks?

3 · The KV cache memory formula, derived term by term

Here is the formula. We will not assert it — we will build it so you can reconstruct it from the picture of what's stored.

$$\text{KV bytes} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times S \times P \times B$$

Term	Meaning	Why it's there
2	K and V	We store two vectors per token, not one
L	number of layers	Each layer has its own independent attention → its own cache
H_{kv}	number of KV heads	K and V are split across heads. This is the term §4 attacks.
d_{head}	dimension per head	Length of each K (and each V) vector
S	sequence length	One K and one V are stored <i>per token</i> in the context
P	precision (bytes)	FP16 = 2. This is the term §5 attacks.
B	batch size	Each concurrent sequence has its own private cache

How to reconstruct it on a whiteboard: "For every layer (L), for every KV head (H_{kv}), I store a K vector and a V vector (the $\times 2$), each of length d_{head} , for every token in context (S), in some precision (P) — and every user in the batch (B) gets their own copy." Read the formula left to right and it narrates itself.

3.1 Worked example — Llama-3-70B, one sequence, FP16

Config: $L = 80$ layers, $H_{\text{kv}} = 8$ KV heads (it uses GQA), $d_{\text{head}} = 128$, $P = 2$ bytes (FP16). Start with **one token** ($S=1$, $B=1$):

Bytes for ONE token (across all layers)

$$\begin{aligned}\text{KV/token} &= 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times P \\ &= 2 \times 80 \times 8 \times 128 \times 2 \\ &= \mathbf{327,680 \text{ bytes} \approx 320 \text{ KB per token}}\end{aligned}$$

Now scale to a full context window of $S = 8,192$ tokens:

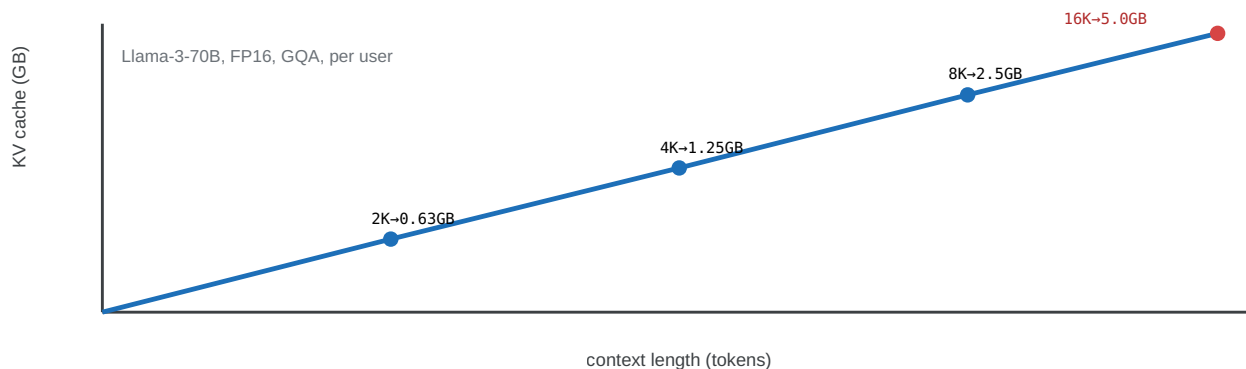
Bytes for ONE full sequence (8K context)

$$\begin{aligned}\text{KV/seq} &= 320 \text{ KB} \times 8,192 \text{ tokens} \\ &= 2,684,354,560 \text{ bytes} \\ &\approx \mathbf{2.5 \text{ GB for a single user at full context}}\end{aligned}$$

There's your "~1 GB per user" — now derived, not guessed. The earlier walkthrough's round number was in the right ballpark; the real figure for 70B at 8K is ~2.5 GB, and now you can see exactly which knobs move it: halve the context → halve it; switch precision → \$5; change the architecture → \$4.

3.2 The scaling insight that catches everyone

KV cache grows **linearly with sequence length and linearly with batch size** — and those multiply. Doubling context *and* doubling concurrent users quadruples cache demand:



Per-user KV cost is a straight line in context length. Long-context products pay for it on every single concurrent request.

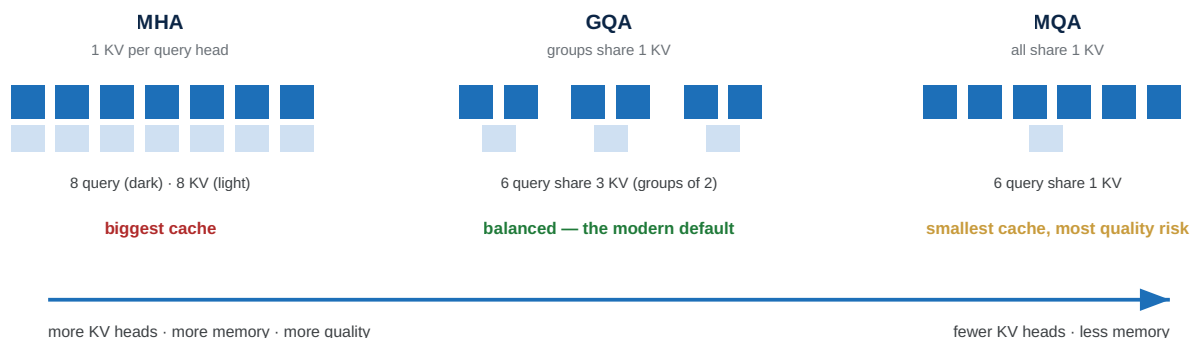
The classic capacity-planning error: sizing a box for "average" context and getting paged out when several users simultaneously hit max context. KV demand is set by the *worst case you allow* ($\text{max_context} \times \text{max_concurrency}$), not the average. We make this concrete with real GPU numbers in §7.

4 · Axis 1 — Architecture: MHA → MQA → GQA

Look back at the formula. The single most powerful lever is H_{kv} , the number of KV heads. Three attention designs are really just three choices for that one term.

4.1 The idea

Attention always uses many **query** heads (say 64) so the model can look for many things at once. The question is: how many distinct **K/V** sets do those query heads need to share?



Dark = query heads (always many). Light = KV heads (the cached ones). The three designs differ only in how many KV sets exist.

4.2 The three designs

- **MHA (Multi-Head Attention)** — the original. Every query head gets its own K/V head. $H_{kv} = H_q$. Maximum quality, maximum cache.
- **MQA (Multi-Query Attention)** — all query heads share *one* K/V head. $H_{kv} = 1$. Tiny cache, but the aggressive sharing can cost quality and training stability.
- **GQA (Grouped-Query Attention)** — the compromise that won. Query heads are split into g groups; each group shares one K/V head. $H_{kv} = g$. Llama-3, Mistral, and Qwen all ship GQA because it captures most of MQA's savings with almost none of the quality loss.

4.3 Same model, three architectures — the numbers

Take the Llama-3-70B shape ($L=80$, $d_{\text{head}}=128$, 64 query heads, $S=8,192$, FP16) and vary *only* H_{kv} :

Design	H_{kv}	KV/token	KV per 8K sequence	vs MHA
MHA	64	2.5 MB	20.0 GB	1×
GQA ($g=8$)	8	320 KB	2.5 GB	8× smaller
MQA	1	40 KB	0.31 GB	64× smaller

Where 20 GB (MHA) comes from

$\text{KV/token} = 2 \times 80 \times 64 \times 128 \times 2 = 2,621,440 \text{ B} \approx 2.5 \text{ MB}$

$\text{KV/seq} = 2.5 \text{ MB} \times 8,192 \approx 20 \text{ GB per user}$ — one user nearly fills an entire H100.

This single table is the whole point of §4. Changing one integer in the config — from 64 to 8 — shrinks per-user memory 8× and turns "1 user per H100" into "8 users per H100" at the same quality. This is why GQA is non-negotiable in modern serving, and why it appears in almost every open model you'll deploy for Anvaya.

Pitfall: GQA savings only apply to the *cache* and to memory-bandwidth during decode — it does **not** shrink the model's weights. A 70B GQA model still has 70B weights (140 GB FP16). People sometimes expect GQA to make the model "smaller to load." It doesn't; it makes the model cheaper to *serve at scale*.

Check yourself: A vendor advertises "70B, 8K context, runs 30 concurrent users on 4×H100." Without their architecture, can you verify the claim? Which single config value do you most need, and why? (Answer in §11.)

5 · Axis 2 — Precision: quantizing the KV cache

The second lever is **P**, the bytes per stored value. You already understand *weight* quantization. KV-cache quantization is the same idea pointed at the cache instead of the weights — and the two are independent, so they stack.

5.1 The savings are mechanical

Halve the bytes per value, halve the cache. Take the GQA 70B result (2.5 GB at FP16, 8K) and just change P:

KV precision	P (bytes)	KV per 8K seq (GQA)	vs FP16	Quality note
FP16 / BF16	2	2.50 GB	1×	Baseline
INT8 / FP8	1	1.25 GB	2× smaller	Usually near-lossless
INT4	0.5	0.63 GB	4× smaller	Small but real degradation

Stacking both axes (the real production move)

Start: MHA, FP16 20.0 GB / user
Apply GQA (g=8): 20.0 / 8 .. **2.50 GB** / user
Apply INT8 KV: 2.50 / 2 **1.25 GB** / user

Net: **16× smaller** than naïve MHA-FP16 – same model, same context.

The axes multiply. Architecture (§4) and precision (§5) are independent knobs: $8\times \times 2\times = 16\times$. This compounding is why a modern stack serves an order of magnitude more users on the same hardware than a 2022-era naïve implementation would.

5.2 Why not always INT4 KV?

Because the K and V values are what the model *attends over* — errors here propagate into every future token's output, and they accumulate over long generations. Weights are quantized once and read; KV errors compound step after step. That's why INT8 KV is the common sweet spot while INT4 KV is reserved for memory-desperate long-context cases.

Pitfall — keys are touchier than values. Keys feed the softmax (an exponential), so a small key error can swing attention weights sharply; value errors only shift the averaged output. Good KV-quantization schemes often keep keys at higher precision than values, or keep a few "sink"/recent tokens in full precision. Treating K and V identically is the naïve mistake.

Check yourself: You quantize weights to INT4 *and* KV cache to INT4 and accuracy collapses, though either alone was fine. Why might the two interact worse than the sum of their parts? (Answer in §11.)

6 · Axis 3 — Memory management: PagedAttention

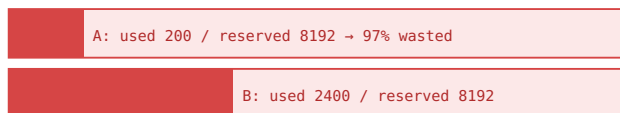
The third lever doesn't shrink the cache per token — it stops you from *wasting* the memory you allocate for it. This is the idea behind vLLM, and it's where a lot of real-world capacity is won or lost.

6.1 The waste problem with naïve allocation

A request *might* generate up to, say, 8,192 tokens, but you don't know how many it actually will. The naïve fix is to reserve a contiguous 8,192-token KV slot up front for every request. Two failures follow:

- **Internal waste:** a request that stops at 200 tokens still holds an 8,192-token reservation — ~97% sits empty but allocated.
- **Fragmentation:** contiguous slots leave odd-sized gaps that no new request fits into, like a parking lot full of single empty spaces but no room for a bus.

Naïve: one big contiguous reservation per request



Free memory exists but is locked inside oversized reservations — new requests get rejected.

PagedAttention: small fixed blocks, allocated on demand



Like OS virtual memory: chop the cache into small fixed blocks (e.g. 16 tokens), hand them out only as a sequence actually grows, and reclaim them instantly when it ends.

6.2 What paging buys you

- **Near-zero internal waste:** a 200-token request holds ~200 tokens' worth of blocks, not 8,192. Waste drops from majority-of-memory to under one block per sequence.
- **No fragmentation:** any free block fits any request — blocks needn't be contiguous, because a lookup table maps logical positions to physical blocks.
- **Prefix sharing (the bonus):** if 50 requests share the same system prompt, the blocks holding that prompt's KV can be stored *once* and pointed to by all 50. A shared 2,000-token system prompt costs one copy, not fifty.

The reframe: §4 and §5 reduce how many bytes a token needs. §6 ensures you actually *use* the bytes you bought. Real systems report naïve allocation wasting 60–80% of KV memory; paging recovers most of it, which directly raises how many users fit. It's a pure-software win with no quality cost.

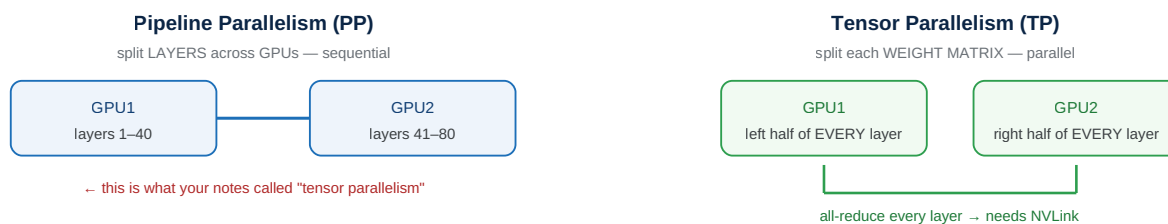
Pitfall: paging isn't free — the lookup indirection adds bookkeeping, and very small block sizes increase overhead while very large blocks bring back internal waste. The block size is a tuning knob (16 tokens is a common default), not a magic constant.

7 · Putting it on GPUs — the full memory budget

Now we reassemble the §0 inequality with real H100 numbers and answer the question the weight formula couldn't: **how many users actually fit?**

7.1 First, fix the parallelism vocabulary

Your earlier notes conflated two different ways of splitting a model. They are not the same, and the difference matters for KV cache:



Both reduce per-GPU weight memory to (total weights ÷ #GPUs).

KV cache is also divided: with TP, each GPU holds the KV for the heads it owns.

TP needs fast intra-node links (NVLink); PP tolerates slower links → used across nodes.

Tensor parallelism splits within layers (heavy communication, intra-node). Pipeline parallelism splits across layers (light communication, cross-node). Total GPUs ≈ TP × PP.

7.2 The budget equation

Per-GPU = (Weights ÷ TP) + (KV cache for its share) + Activations + Overhead

Rearranged, the memory left for users is what sets concurrency:

Free for KV = Total VRAM – Weights – Activations – Overhead

Max concurrent users ≈ Free for KV ÷ (KV per user at max context)

7.3 Worked example — 70B on 2×H100 vs 4×H100

Model: Llama-3-70B, FP16, GQA. Weights = 140 GB. KV per user at 8K = 2.5 GB (from §3). Reserve ~10% per GPU for activations + framework overhead.

Setup A — 2 × H100 (160 GB total), TP = 2

Total VRAM 160 GB

– Weights 140 GB

– Overhead (~10%) 16 GB

= Free for KV **~4 GB**

Users = 4 ÷ 2.5 ≈ **1–2 concurrent users at full 8K context**

Setup B — 4 × H100 (320 GB total), TP = 4

Total VRAM 320 GB

– Weights 140 GB

– Overhead (~10%) 32 GB

= Free for KV **~148 GB**

Users = $148 \div 2.5 \approx$ **~59 concurrent users at full 8K context**

This is the punchline of the whole document. Going from 2 to 4 H100s only ~1.14×'d the weight headroom, but it took usable concurrency from ~2 users to ~59 — a **~30× jump** — because the marginal VRAM went almost entirely to KV cache. Providers run TP=4 or TP=8 even when the weights "fit" in TP=2, purely to buy KV headroom. The weight formula alone would have told you both setups "work." Only the KV math reveals one is nearly useless for serving.

Pitfall: if users rarely hit full 8K, you can safely oversubscribe — paging (§6) means idle/short sequences free their blocks. But you must cap admission so a burst of long-context requests can't blow past VRAM mid-generation, which triggers either request preemption or an out-of-memory crash. Capacity is governed by the *distribution* of context lengths, not a single number.

Check yourself: In Setup B, what happens to the ~59-user limit if you switch KV to INT8 (§5)? And separately, if you raised max context from 8K to 32K? Compute both. (Answer in §11.)

8 · Prefill vs decode — why decode is the bottleneck

One last concept ties the KV cache to *speed*. Inference has two phases with opposite hardware profiles:

Phase	What happens	Bound by	KV cache role
Prefill	Process the whole input prompt at once	Compute (one big parallel matmul, GPU saturated)	Writes the initial cache for all prompt tokens
Decode	Generate one new token, repeat	Memory bandwidth (stream all weights + full cache per token)	Reads the entire cache every single step

Why decode is memory-bound, in one line: to emit *one* token, the GPU must pull the *entire* weight set and the *entire* KV cache out of HBM — billions of bytes moved to do relatively little math. Throughput is set by HBM bandwidth, not FLOPs. This is the low arithmetic-intensity regime of the roofline model.

Where "~200 tokens/sec" actually comes from

70B FP16 on 2×H100, TP=2 → each GPU streams ~70 GB of weights/token.

H100 HBM bandwidth ≈ 3.35 TB/s.

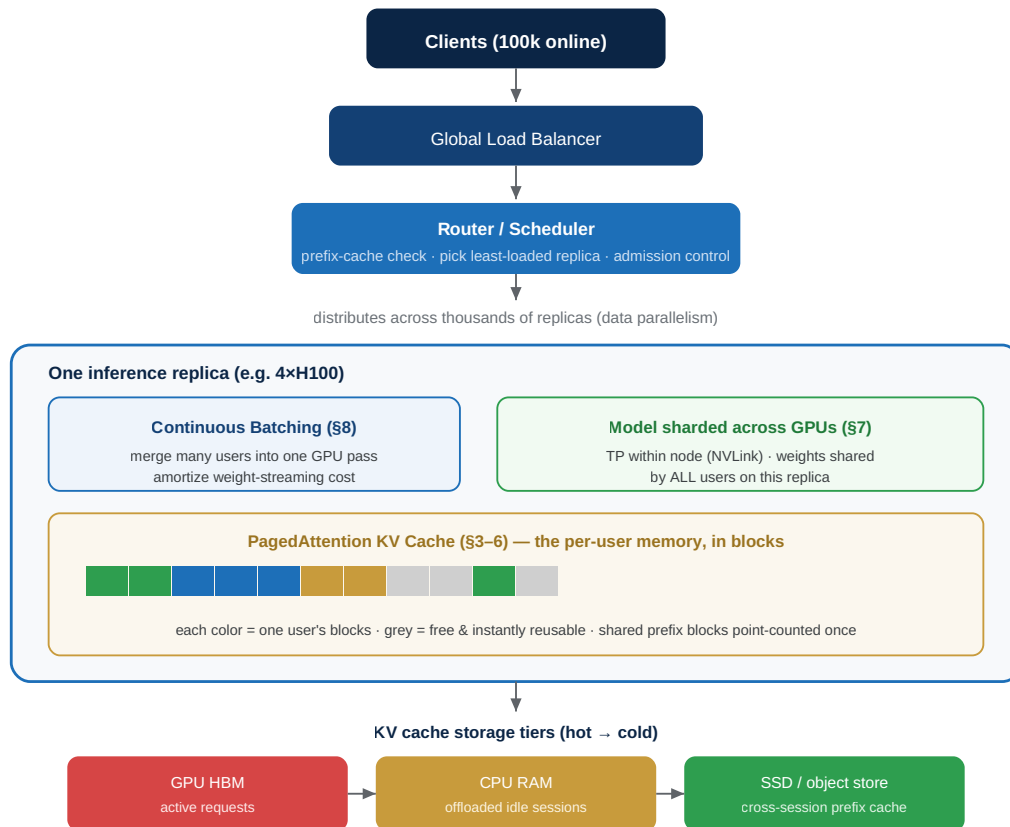
time/token ≈ 70 GB ÷ 3,350 GB/s ≈ **21 ms** → **~48 tok/s for a SINGLE stream**

Batching amortizes that weight-streaming cost across many users in one pass, pushing aggregate throughput into the **hundreds of tok/s** — exactly the ballpark your earlier walkthrough assumed. Now you know it's a bandwidth calculation, not a guess.

This closes the loop with continuous batching. Because the weight-streaming cost is paid *per step regardless of batch size*, running 1 user vs 64 users costs almost the same HBM traffic for weights — so you amortize it across as many concurrent sequences as your KV cache budget (\$7) allows. KV headroom and decode throughput are the same constraint viewed two ways.

9 · The complete production architecture

Every concept now slots into one picture. This is what's behind the "your request isn't assigned a private GPU" takeaway from your earlier walkthrough — now you can name every box and why it exists.



End-to-end serving. Weights are shared by everyone on a replica; only the colored KV blocks are per-user. That asymmetry — shared weights, private cache — is what lets thousands of users share one model.

Anvaya parallel. That hot → warm → cold KV tiering is structurally the same problem as the working / episodic / semantic memory tiers in your own architecture: what's expensive to keep instantly accessible vs. cheap to fetch on demand. Same caching logic, different domain — the eviction and promotion policies rhyme.

10 · The "explain it to anyone" cheat sheet

Everything compressed to what fits on a whiteboard. If you can reproduce this page from memory, you can teach the whole topic.

Three formulas

1. **Weights:** $\text{Weight bytes} = N_{\text{params}} \times \text{bytes_per_param}$
2. **KV cache:** $\text{KV bytes} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times S \times P \times B$
3. **Capacity:** $\text{Max users} \approx (\text{VRAM} - \text{Weights} - \text{Overhead}) \div (\text{KV per user})$

Three KV levers (they multiply)

Axis	Knob	Move	Typical win
Architecture	H_{kv}	MHA → GQA	~8× smaller cache
Precision	P	FP16 → INT8	2× smaller cache
Memory mgmt	allocation	naïve → PagedAttention	recover 60–80% wasted

Three analogies that make it click

- **Weights = kitchen, KV cache = tables.** Big kitchen, full tables → you still turn people away. Capacity is table-counting.
- **KV cache = a notebook you write once and reread.** Without it you'd re-read the whole book to write each new sentence ($O(n^2)$); with it you flip to your notes ($O(n)$).
- **PagedAttention = OS virtual memory.** Small blocks on demand beat one giant reservation per program.

The one-paragraph version

An LLM server loads *one* set of weights shared by everyone. To avoid recomputing attention, it stores every token's Key and Value vectors — the KV cache — which is *private* to each conversation and grows with context length. That per-user cache, not the shared weights, is what caps how many people a GPU can serve. You shrink it three independent ways — fewer KV heads (GQA), fewer bytes per value (quantization), and smarter allocation (PagedAttention) — and they multiply. Generation is bottlenecked by streaming weights and cache out of memory each token, so servers batch many users into each pass to share that cost. Scale out by replicating this whole unit behind a load balancer.

11 · Answers to the self-test questions

§2 — Why not cache Q?

Because K and V of a past token are *reused* by every future token's attention, so storing them saves repeated work. A past token's Q is used only at the single step that token was generated and is never referenced again — caching it would store something never reread. Cacheability comes from "will this be read again?", and only K and V will be.

§4 — Verifying the vendor's "30 users on 4×H100" claim

The config value you most need is H_{kv} (number of KV heads / GQA group count), because it sets KV-per-user, which dominates the capacity calculation. With 4×H100 (320 GB) – 140 GB weights – overhead $\approx$ 148 GB free. At GQA-8 (2.5 GB/user, 8K) that's $\sim$ 59 users — so 30 is plausible with headroom. If it were MHA (20 GB/user), you'd fit only $\sim$ 7 — and the claim would be false. One integer flips the verdict.

§5 — Why INT4 weights AND INT4 KV interact badly

Errors compose along the same compute path. Quantized weights produce slightly-off K and V; storing those *already-approximated* values in INT4 adds a second rounding on top. During decode, the error then re-enters attention every step and accumulates over the generation. Each source was individually tolerable, but stacked-and-compounded-over-time they cross the threshold where attention picks wrong tokens. Quantization errors are multiplicative through the pipeline, not additive.

§7 — Setup B under INT8 KV, and under 32K context

INT8 KV: per-user halves 2.5 $\rightarrow$ 1.25 GB. Users = $148 \div 1.25 \approx \sim$ **118 users** (doubled).
32K context (FP16): per-user $4\times \rightarrow 2.5 \times 4 = 10$ GB. Users = $148 \div 10 \approx \sim$ **14 users** (quartered).
Both together (INT8 + 32K): per-user = 5 GB $\rightarrow 148 \div 5 \approx \sim$ **29 users**.

This is the whole document in one exercise: the three levers and the two scaling dimensions, all moving the same capacity number predictably.

Scope per request: KV cache types limited to MHA / MQA / GQA + quantization + PagedAttention; examples anchored to H100-class datacenter hardware. Advanced types (MLA/DeepSeek, eviction, cross-layer reuse, disaggregated serving à la Mooncake) were deliberately left out and are the natural next layer if you want a follow-up. Model configs reflect Llama-3-70B's published shape; GPU-fleet figures from the source conversation are illustrative estimates, not official disclosures.